JIHANG LI

Irvine, California 92612 | (510) 766-4360 | lijihang21@gmail.com | linkedin.com/in/Jihang-li21 | https://jihangli1121.github.io/My-website/

SUMMARY

- Master of Computer Science student conducting 3D cell-lineage reconstruction research (CellUniverse), designing Monte Carlo optimization and PCA-based cell-division detection algorithms in C++ for biological imaging.
- Software engineer at startup LaunchPath, building an AI-powered sales-outreach email platform end-to-end — Spring Boot and React, RAG-grounded email generation with pgvector, and autonomous LLM agent workflows.
- Quickly learn and master new technologies; successful working in both team and self-directed settings.

EDUCATION

UNIVERSITY OF CALIFORNIA, IRVINE, Irvine, California
Master of Computer Science, Expected December 2026, GPA: 3.97/4.00

UNIVERSITY OF CALIFORNIA, SANTA CRUZ, Santa Cruz, California
B.S., Computer Science, 09/2021 – 12/2024, GPA: 3.67/4.00

TECHNICAL SKILLS

Languages:	Java, Python, C/C++, JavaScript, HTML/CSS
Frameworks:	Node.js, Django, REST API
Developer Tools:	Git, Google Cloud Platform, VS Code, Visual Studio, PyCharm, IntelliJ, Jupyter Notebook
Libraries:	Pandas, NumPy, Matplotlib, PyTorch, TensorFlow

EXPERIENCE

LAUNCHPATH, Remote, California
Software Engineer, LaunchMail — AI sales-outreach email platform **Jun 2026 - Present**

- Shipped LaunchMail, an AI sales-email platform (Spring Boot, React/TypeScript, PostgreSQL) with 110+ REST endpoints.
- Implemented RAG-grounded AI email drafting with pgvector embeddings, unified across 5 LLM providers.
- Built autonomous outreach agents and a drag-and-drop workflow engine on a webhook-resumed Postgres state machine.
- Engineered deliverability safeguards: sender rotation, randomized throttling, daily caps, and a 220-term spam linter.
- Ensured reliability with 242 integration tests (Testcontainers, WireMock) across 33 Flyway schema migrations.

UNIVERSITY OF CALIFORNIA, SANTA CRUZ, Santa Cruz, California
Undergraduate Researcher, AIEA Lab, Research under Dr. Leilani Gilpin **Jan 2024 - Jun 2025**

- Research Gaussian splatting techniques to reconstruct dangerous scenes, enhancing vehicle response to hazard situations.
- Conducted object detection research to improve recognition in complex urban areas, using machine learning algorithms.
- Utilized Python, TensorFlow, and PyTorch for experiments, data analysis, and model development.
- Collaborated with a multidisciplinary team to advance state-of-the-art research in autonomous driving technology.
- Improved Gaussian-splatting rendering efficiency by 30%, accelerating playback of reconstructed hazard scenes for analysis.

PROJECTS

CellUniverse 3D Cell Tracking ● *Software Engineer* ● Algorithm ● C++ ● Github **Feb 2026 - Present**
MCS Keystone & Capstone research lead by Dr. Wayne Hayes's focusing on 3D cell lineage reconstruction.

- Replaced voxel-matrix rendering with analytical inverse-rotation ellipsoid testing, reducing memory and fixing artifacts.
- Built Monte Carlo optimizer fitting 3D oblate spheroids to confocal microscopy data via L2 cost minimization.
- Developed PCA-based cell division detection clustering real-image bright pixels to find split axes and daughter positions.
- Implemented per-axis normalization and neighbor-exclusion filtering to prevent PCA contamination from adjacent cells.
- Designed two-phase optimization separating cell fitting from split detection to prevent cascading false divisions.
- Contributing to a research manuscript on automated 3D cell lineage tracking across biological imaging datasets.

Doom RL Agent ● *Machine Learning Engineer* ● Reinforcement Learning ● Python ● Github **Sep 2025 - Dec 2025**
Collaborative RL project training PPO agents to play the 1993 Doom via Vizdoom scenarios

- Implemented PPO agents via Stable-Baselines3 targeting three VizDoom scenarios with increasing enemy complexity.
- Designed a reward shaping system with survival bonuses, death penalties, and kill rewards to guide exploration.
- Applied running-statistics reward normalization to stabilize gradients and accelerate policy convergence.
- Tuned PPO hyperparameters (learning rate decay, batch size, entropy coefficient) improving survival from ~38 to 300+ steps.
- Containerized training with a GPU-accelerated Docker environment using CUDA 11.8 and docker-compose.